

Aurora Siger — Fase 4: SIGIC — Sistema Inteligente de Gerenciamento da Infraestrutura da Colônia

Relatório Técnico Integrador

Gabriel Carmona Bittencourt · Lúri Leão de Almeida · Márcio Francisco dos Santos Júnior · Maria Sophia Domingues dos Santos

2026

Aurora Siger — Fase 4: SIGIC

Atividade Integradora — Ciência da Computação (online), FIAP, 2026 Repositório: <https://github.com/iurileo-hub/FIAP-Aurora-Siger>

Nota de procedência e continuidade

A Fase 4 foi desenvolvida pela equipe como entrega autônoma. Esta versão integra o resultado ao portfólio monorrepo FIAP-Aurora-Siger, sendo uma **consolidação** realizada por Lúri Leão de Almeida que mantém os 13 módulos da Fase 3 como nós canônicos do grafo, deriva as prioridades operacionais diretamente da árvore de criticidade da Fase 3 (sem duplicar tabelas) e expõe o sistema via CLI `sigic`.

Autores da equipe:

Nome	RM
Gabriel Carmona Bittencourt	RM569239
Márcio Francisco dos Santos Júnior	RM570758
Lúri Leão de Almeida	RM570215
Maria Sophia Domingues dos Santos	RM571209

A curadoria e síntese desta consolidação são de Lúri; todos os quatro autores permanecem creditados pelo trabalho coletivo da equipe.

Resumo

O SIGIC representa computacionalmente a infraestrutura da colônia Aurora Siger como um **grafo ponderado não-dirigido** de 13 nós e 20 arestas tipadas. Sobre essa estrutura implementam-se BFS por níveis, DFS iterativo, Dijkstra com restrição de prioridade, detecção de pontos de articulação pelo algoritmo de Tarjan e centralidade de intermediação pelo método de Brandes. A análise revela que o **Armazenamento e Logística** (módulo 9, tier Expansão) é o único ponto de corte da rede — um único ponto de falha real apesar de classificação de baixa prioridade — e que o **Reator Nuclear** (módulo 5) possui a maior centralidade de intermediação (0,2879). A modelagem matemática via $C(t) = C_0 e^{rt}$ projeta que o consumo atingirá 90 % da geração instalada (210 kW) em aproximadamente **7,1 anos** a partir de 2026.

1. Organização da infraestrutura da colônia

1.1 Os treze módulos — continuidade da Fase 3

A colônia opera com **13 módulos** herdados diretamente da Fase 3. Módulos 1–12 existiam desde o pouso (Fase 2); o módulo 13 (Gerador Eólico) foi incorporado na Fase 3. A Fase 4 não redefine essa lista: reutiliza `aurora_siger.operations.MODULES` como fonte única de verdade para identidade, tipo e consumo.

Cada nó do grafo é um `Module` (dataclass) com: `id`, `name`, `type`, `consumption` (kW no modo “adequate”), `priority` (1–10, derivada do tier de criticidade da Fase 3), `capacity` (kWh de armazenamento local), `communication_need` (1–10) e `position` (coordenadas no layout bidimensional).

#	Módulo	Tipo	Criticidade	Prioridade	Consumo (kW)	Grau
1	Controle e Comando	consumer	Vital	10	8,0	4
2	Suporte de Vida (ECLSS)	consumer	Vital	10	12,0	3
3	Habitat	consumer	Vital	10	15,0	5
4	Energia Solar	solar_generator	Sustento	7	1,0	3
5	Reator Nuclear	nuclear_generator	Sustento	7	3,0	5
6	Comunicações	consumer	Sustento	7	5,0	2
7	Suporte Médico	consumer	Vital	10	6,0	3
8	Produção de Alimentos	consumer	Sustento	7	10,0	3
9	Armazenamento e Logística	consumer	Expansão	4	4,0	4
10	ISRU (Recursos Locais)	consumer	Sustento	7	8,0	2
11	Oficina e Manutenção	consumer	Expansão	4	3,0	3
12	Laboratório Científico	consumer	Expansão	4	5,0	2
13	Gerador Eólico	wind_generator	Sustento	7	0,5	1

Tabela 1 — Módulos da colônia Aurora Siger: atributos e posição topológica Fonte: Elaborada pelos autores (2026)

A prioridade de cada módulo é derivada automaticamente da árvore de criticidade da Fase 3: Vital → 10, Sustento → 7, Expansão → 4. Isso garante que a mesma lógica de emergência da Fase 3 (rebaixar Expansão antes de Sustento; Vital nunca desliga) seja coerente com o roteamento prioritário do SIGIC na Fase 4.

1.2 Geração instalada

A colônia dispõe de **210 kW de geração instalada**:

- Solar (módulo 4): 100 kW de capacidade máxima
- Nuclear (módulo 5): 80 kW de capacidade máxima
- Eólico (módulo 13): 30 kW de capacidade máxima

O consumo total no modo adequado é $C_0 = 80,5$ kW — soma direta dos 13 módulos —, representando 38,3 % da geração instalada em 2026.

2. Representação da rede em grafos

2.1 Estrutura do grafo

A infraestrutura é modelada como um **grafo não-dirigido e ponderado** $G = (V, E)$ onde:

- V = conjunto de 13 módulos (vértices)
- E = conjunto de 20 conexões (arestas) com peso $w \in \{1, 2, 3\}$

Os pesos representam o custo da conexão: distância física, atenuação do sinal ou custo energético de transmissão. Valores menores indicam conexões mais eficientes.

As arestas são tipadas em três categorias operacionais:

Tipo	Quantidade	Significado
energy	11	Fluxo de potência elétrica entre módulos
data	6	Troca de informação e controle
life	3	Fluxo de suprimentos vitais (ar, água)

Tabela 2 — Tipos de conexão na rede da colônia Fonte: Elaborada pelos autores (2026)

2.2 Lista completa de arestas

Módulo A	Módulo B	Peso	Tipo
5 — Reator Nuclear	1 — Controle e Comando	2	energy
5 — Reator Nuclear	2 — Suporte de Vida	2	energy
5 — Reator Nuclear	3 — Habitat	3	energy
4 — Energia Solar	3 — Habitat	2	energy
4 — Energia Solar	8 — Produção de Alimentos	2	energy
4 — Energia Solar	9 — Armazenamento e Logística	2	energy
5 — Reator Nuclear	9 — Armazenamento e Logística	2	energy
13 — Gerador Eólico	9 — Armazenamento e Logística	3	energy
2 — Suporte de Vida	3 — Habitat	1	life
3 — Habitat	7 — Suporte Médico	1	life
2 — Suporte de Vida	7 — Suporte Médico	2	life
1 — Controle e Comando	6 — Comunicações	2	data
1 — Controle e Comando	12 — Laboratório Científico	3	data
1 — Controle e Comando	3 — Habitat	2	data
6 — Comunicações	7 — Suporte Médico	3	data
8 — Produção de Alimentos	10 — ISRU	2	energy
10 — ISRU	11 — Oficina e Manutenção	2	data
11 — Oficina e Manutenção	9 — Armazenamento e Logística	2	energy
12 — Laboratório Científico	8 — Produção de Alimentos	2	data
11 — Oficina e Manutenção	5 — Reator Nuclear	3	energy

Tabela 3 — Arestas da rede (grafo não-dirigido ponderado) Fonte: Elaborada pelos autores (2026)

2.3 Justificativa da topologia

A topologia reflete decisões operacionais concretas:

1. **Reator Nuclear (5) e Energia Solar (4) como geradores centrais:** concentram conexões de energia para os módulos de maior consumo (Habitat, Suporte de Vida, Alimentos). O Reator Nuclear (grau 5) é o hub energético principal — evidência corroborada pela sua centralidade de intermediação de 0,2879.
2. **Habitat (3) como hub de suprimentos vitais:** conectado por arestas do tipo `life` ao Suporte de Vida e ao Suporte Médico, pois o ambiente habitável é o ponto de convergência dos fluxos vitais (ar, água, calor).
3. **Controle e Comando (1) como hub de dados:** conectado a Habitat, Comunicações e Laboratório por arestas `data`, pois toda instrução operacional e telemetria passa pelo centro de controle.
4. **Armazenamento e Logística (9) como intermediário de energia secundária:** recebe dos geradores Solar (#4), Nuclear (#5) e Eólico (#13) e redistribui via Oficina e Manutenção (#11). Sua posição como único elo do Eólico torna-o um **ponto de articulação** (ver Seção 4.2).
5. **Gerador Eólico (13) como folha:** conectado exclusivamente ao Armazenamento e Logística — topologia de folha justificada pela localização periférica do gerador eólico (fora do núcleo habitado) e pela necessidade de agregação via depósito antes da redistribuição.

2.4 Diagrama da rede

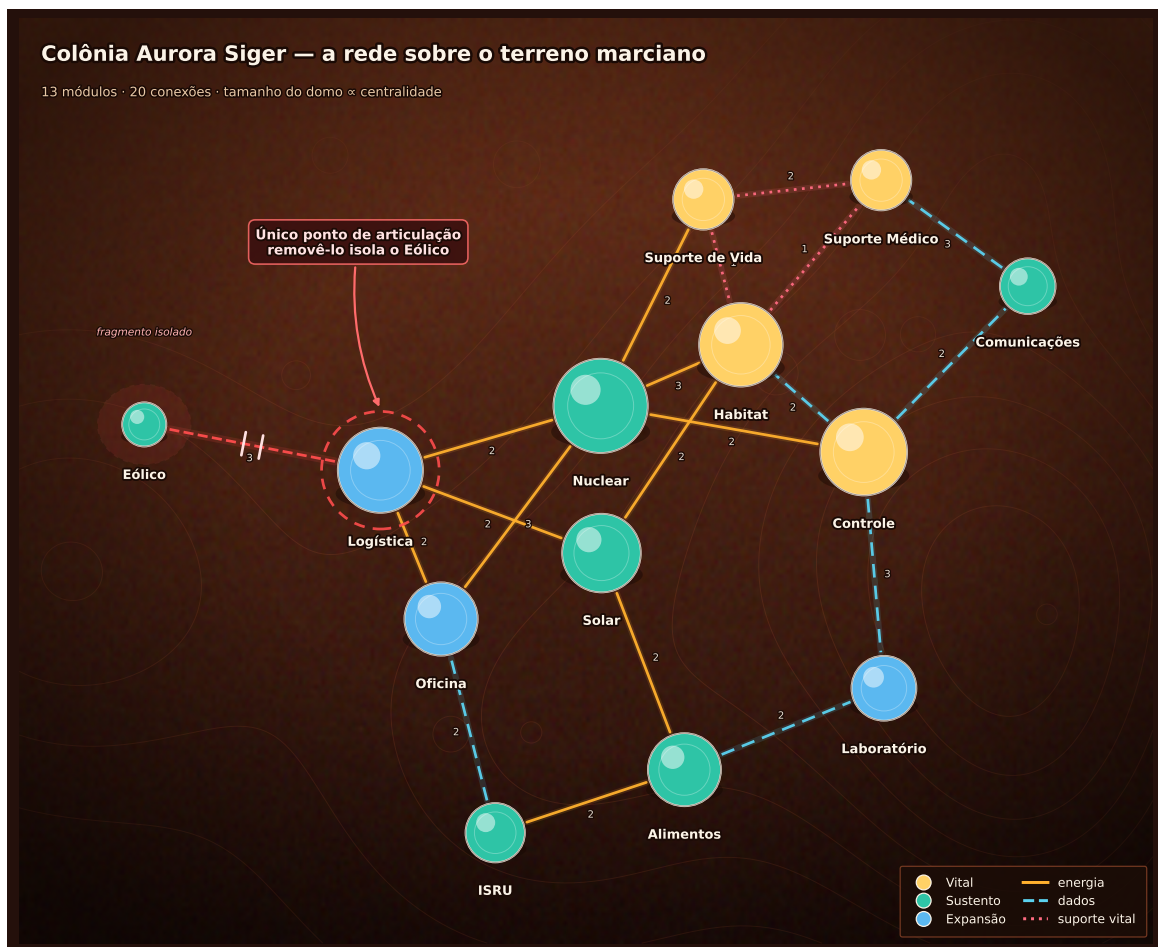


Figura 1: Mapa da colônia Aurora Siger sobre o terreno marciano — 13 módulos (domos) e 20 conexões (dutos); o ponto de articulação e o Gerador Eólico isolado aparecem destacados.

O mapa acima foi gerado por `figuras/gerar_rede.py` (matplotlib). Os nós são coloridos por tier de criticidade (Vital, Sustentente, Expansão) e dimensionados pela centralidade de intermediação (Seção 3.5); as arestas

seguem o tipo de conexão (energia, dados, suporte vital). O layout é force-directed (Fruchterman-Reingold) determinístico, iniciado nas posições físicas dos módulos. O Gerador Eólico (#13) aparece isolado na periferia, ligado à rede apenas pelo Armazenamento e Logística (#9) — o ponto de articulação destacado em vermelho.

3. Implementação dos algoritmos de redes

3.1 BFS — Busca em largura por níveis

`search.bfs(graph, start, target)` usa fila (`collections.deque`) e explora o grafo nível a nível a partir do nó de origem. Para cada nó visitado registra o nível de distância (BFS-hop) e o caminho desde a origem. Se `target` for fornecido, retorna o nível em que foi encontrado.

Aplicação na colônia: mapear todos os módulos acessíveis a partir de um ponto de emergência, respeitando a distância em saltos (não em peso). Útil para triagem rápida de alcançabilidade — ex.: quantas estações separam o Controle e Comando do Suporte Médico em condição de falha parcial da rede.

Resultado típico (BFS a partir do módulo 1): - Nível 0: [1] — Controle e Comando - Nível 1: [5, 6, 12, 3] — vizinhos diretos - Nível 2: [2, 9, 11, 7, 8, 4] — dois saltos - Nível 3: [13, 10] — três saltos

3.2 DFS — Busca em profundidade

`search.dfs(graph, start, target)` implementa DFS iterativo com pilha explícita (evita estouro de recursão), retornando a ordem de visita e, se `target` fornecido, o caminho raiz→alvo. Ao contrário do BFS, DFS mergulha num ramo antes de explorar outros — adequado para detectar caminhos alternativos e verificar conectividade.

`search.connected_components(graph)` usa DFS para enumerar componentes conectados. No grafo atual, todos os 13 módulos pertencem a **um único componente** — a rede é completamente conectada.

3.3 Dijkstra — Caminho mínimo ponderado

`paths.shortest_path(graph, origin, destination)` implementa o algoritmo de Dijkstra com heap mínimo (heapq). A variante `paths.shortest_path_with_priority` aceita um parâmetro `min_priority` que descarta vizinhos com prioridade abaixo do limiar durante a relaxação — permitindo, por exemplo, rotear energia apenas por módulos de Sustento ou superior (prioridade maior ou igual a 7), ignorando módulos de Expansão em falha.

Exemplo canônico (Dijkstra de Energia Solar #4 para Suporte Médico #7):

- Sem restrição: $4 \rightarrow 3 \rightarrow 7$, custo 3 (peso 2 + peso 1)
- Com restrição prioridade maior ou igual a 7: mesmo resultado (nós intermediários têm prioridade maior ou igual a 7)

`paths.all_shortest_paths(graph, origin)` computa os caminhos mínimos para todos os demais módulos a partir de uma origem — utilizado pelo CLI para exibir a “árvore de distribuição” a partir de qualquer gerador.

3.4 Pontos de articulação — Algoritmo de Tarjan

`analysis.articulation_points(graph)` implementa a DFS de Tarjan para identificar vértices de corte (*cut vertices*). Um vértice u é ponto de articulação se:

- é raiz da DFS e tem mais de um filho; **ou**
- não é raiz e existe filho v tal que $\text{low}[v] \geq \text{disc}[u]$ (nenhum descendente de v tem back-edge para ancestral de u).

Resultado no grafo da colônia:

Único ponto de articulação: módulo 9 — Armazenamento e Logística

Remover o módulo 9 isolaria o Gerador Eólico (#13) do restante da rede. Ainda que o Gerador Eólico represente apenas 30 kW de capacidade e o módulo 9 seja classificado como Expansão (prioridade 4), a topologia expõe um risco real: a geração eólica torna-se inacessível à rede sem esse nó intermediário. A descoberta algorítmica contradiz a intuição baseada apenas na criticidade operacional e ilustra o valor da análise topológica.

3.5 Centralidade de intermediação — Algoritmo de Brandes

`analysis.betweenness(graph)` implementa o algoritmo de Brandes para calcular a centralidade de intermediação de cada nó — a fração dos caminhos mais curtos entre todos os pares de módulos que passa pelo nó. O valor é normalizado para $[0, 1]$ dividindo por $(n - 1)(n - 2)$.

Módulo	Centralidade de intermediação
5 — Reator Nuclear	0,2879
1 — Controle e Comando	0,2083
9 — Armazenamento e Logística	0,1932
3 — Habitat	0,1780
4 — Energia Solar	0,1364
11 — Oficina e Manutenção	0,0972
8 — Produção de Alimentos	0,0960
12 — Laboratório Científico	0,0492
2 — Suporte de Vida	0,0303
7 — Suporte Médico	0,0303
10 — ISRU	0,0265
6 — Comunicações	0,0152
13 — Gerador Eólico	0,0000

Tabela 4 — Centralidade de intermediação de Brandes (normalizada) Fonte: Elaborada pelos autores (2026)

O Reator Nuclear lidera com 0,2879 — confirma seu papel de hub energético central. O Gerador Eólico tem centralidade zero: como folha (grau 1), nenhum caminho mais curto entre dois módulos distintos passa por ele. Isso distingue relevância topológica (intermediação) de relevância operacional (capacidade de geração).

Nota metodológica: a implementação de Brandes substituiu uma abordagem anterior baseada em enumeração exaustiva de todos os caminhos simples (complexidade exponencial). Brandes opera em $O(V \cdot E)$ — polinomial, determinístico e adequado a grafos maiores.

3.6 Coeficiente de agrupamento

`analysis.clustering_coefficient(graph)` calcula o coeficiente médio de agrupamento local: para cada nó com grau maior ou igual a 2, mede a fração de pares de vizinhos que estão diretamente conectados entre si.

Resultado: coeficiente médio = **0,189** — indica que a rede tem baixa tendência de formar triângulos, o que é esperado numa infraestrutura de colônia onde as conexões são funcionais (não redundantes por padrão) e poupam recursos.

4. Estruturas de dados em Python

4.1 Lista de adjacência — `dict[int, list[int]]`

O quê: para cada módulo (chave inteira), uma lista dos ids dos módulos vizinhos.

Por quê: a lista de adjacência é eficiente em espaço quando o grafo é esparsos — 20 arestas em 13 nós produzem densidade $2e/n(n-1) \approx 0,26$. BFS, DFS e Dijkstra iteram sobre vizinhos em tempo $O(\text{grau})$ por nó, sem varrer colunas zeradas.

4.2 Matriz de adjacência — `list[list[float]]`

O quê: matriz $n \times n$ onde `matrix[i][j]` = peso da aresta (i, j) ou 0 se não houver conexão.

Por quê: permite consulta de peso em $O(1)$ por índice, sem busca em dicionário. Útil para análises matriciais e visualização tabular da rede. A implementação mantém as duas representações sincronizadas via `_rebuild_matrices()` chamado a cada inserção.

4.3 Dicionário de pesos — `dict[str, float]`

O quê: `edge_weights` mapeia a chave canônica " $\min(a,b)-\max(a,b)$ " ao peso da aresta.

Por quê: o grafo é não-dirigido; a chave canônica garante que (a, b) e (b, a) acessem o mesmo peso sem duplicar entradas. Dicionário oferece lookup $O(1)$ médio.

4.4 Dicionário de tipos de conexão — `dict[str, str]`

O quê: `connection_types` usa a mesma chave canônica e armazena o tipo (`energy/data/life`).

Por quê: separar o tipo do peso mantém cada atributo em sua estrutura mais adequada. Filtragem por tipo (ex.: mostrar apenas arestas `energy`) é $O(E)$ sem interferir nos pesos.

4.5 Dataclass `Module` — tupla nomeada com mutabilidade controlada

O quê: `@dataclass class Module` agrupa os atributos fixos de cada nó. Funciona como uma tupla nomeada extensível — imutabilidade por convenção (não por `frozen`), mas campos acessíveis por nome.

Por quê: `dataclass` oferece `__repr__` automático, comparação por campo, tipagem estática e é mais legível que um `dict` puro. A escolha não-frozen permite atualizar `status` em runtime (ex.: módulo entra em manutenção) sem recriar o objeto.

4.6 Tuplas nas arestas de topologia — `list[tuple[int, int, float, str]]`

O quê: `topology.EDGES` é uma lista de tuplas (`id1, id2, peso, tipo`).

Por quê: tuplas são imutáveis — a topologia da colônia é declarativa, não deve ser alterada em runtime. A imutabilidade comunica a intenção ao leitor do código e previne modificações acidentais.

5. Modelagem matemática e otimização

5.1 Função de crescimento do consumo

O consumo energético total da colônia é modelado pela função exponencial:

$$C(t) = C_0 \cdot e^{rt}$$

onde:

- $C_0 = 80,5$ kW — consumo inicial (soma dos 13 módulos no modo adequado)
- $r = 0,12$ ano⁻¹ — taxa de crescimento anual (12 %)
- t — tempo em anos a partir de 2026

A escolha do modelo exponencial é fundamentada: o crescimento de consumo em colônias em expansão segue padrão multiplicativo — cada novo módulo adicionado eleva a base de consumo sobre a qual o crescimento seguinte incide. O modelo captura esse efeito de composição.

5.2 Análise por cálculo diferencial

Primeira derivada (taxa instantânea de crescimento):

$$C'(t) = r \cdot C_0 \cdot e^{rt} = r \cdot C(t)$$

Em $t = 0$: $C'(0) = 0,12 \times 80,5 = 9,66$ kW/ano

A taxa de crescimento é proporcional ao próprio consumo — propriedade característica do crescimento exponencial. A cada ano adicional, a colônia consome mais 9,66 kW *no primeiro ano*, e um valor crescente nos anos seguintes.

Segunda derivada (aceleração do crescimento):

$$C''(t) = r^2 \cdot C_0 \cdot e^{rt} > 0 \quad \forall t$$

A segunda derivada é sempre positiva: a curva é sempre convexa — o crescimento é sempre acelerado. Não existem pontos críticos (máximos ou mínimos locais) no interior do domínio, pois $C'(t) \neq 0$ para qualquer t finito.

Diferenciação numérica: a implementação usa diferença central:

$$C'(t) \approx \frac{C(t+h) - C(t-h)}{2h}, \quad h = 0,001$$

A diferença central tem erro de truncamento $O(h^2)$, superior à diferença progressiva $O(h)$, garantindo precisão adequada sem custo computacional relevante.

5.3 Ponto crítico de capacidade

Define-se o **ponto crítico de geração** como o instante t^* em que o consumo projetado atinge 90 % da geração instalada (210 kW):

$$C(t^*) = 0,9 \times 210 = 189 \text{ kW}$$

Resolvendo:

$$t^* = \frac{\ln(189/80,5)}{0,12} \approx 7,1 \text{ anos}$$

Em 2033, com crescimento de 12 % ao ano, a demanda da colônia chegará a 189 kW — 90 % dos 210 kW instalados. Esse é o **horizonte de planejamento crítico**: antes de 2033, a Aurora Siger precisa ampliar a capacidade de geração ou reduzir a taxa de crescimento do consumo por meio de eficiência energética.

A `predict_critical_point()` verifica esse limiar numericamente com passo de 0,5 ano — confirmando $t^* \approx 7,1$ para o cenário padrão ($r = 0,12$).

5.4 Perda energética por distância

As conexões entre módulos dissipam energia. O modelo de perda por distância é:

$$P_{\text{loss}} = 1 - e^{-d(1-\eta)}$$

onde d é o peso da aresta (distância normalizada) e $\eta = 0,95$ é a eficiência de transmissão (95 %).

Peso da aresta (d)	Perda estimada
1	4,88 %
2	9,52 %
3	13,93 %

Tabela 5 — Perda energética por peso da conexão Fonte: Elaborada pelos autores (2026)

Conexões com peso 1 (life: Suporte de Vida – Habitat e Habitat – Suporte Médico) são as mais eficientes — coerente com sua criticidade vital. As conexões de dados de longa distância (peso 3) incorram nas maiores perdas, mas trocam informação, não potência bruta.

5.5 Cenários de crescimento

Três cenários examinam a sensibilidade ao parâmetro r :

Cenário	Taxa r	Consumo em 10 anos	Status
Otimista	8 %	179,2 kW	Atenção (> 168 kW = 80 % de 210 kW)
Moderado	12 %	267,3 kW	Crítico (> 210 kW em ~7 anos)
Pessimista	18 %	487,0 kW	Crítico (supera geração antes de 5 anos)

Tabela 6 — Cenários de crescimento do consumo energético Limiares: > 210 kW → CRÍTICO; > 168 kW (80 % de 210 kW) → ATENÇÃO; ≤ 168 kW → SEGURO. Fonte: *MathematicalModeling.simulate_scenarios()* (2026)

O cenário moderado — mais provável para uma colônia em expansão — já ultrapassa a capacidade instalada em 10 anos. Isso reforça a urgência de ampliar geração ou adotar eficiência energética antes do horizonte crítico de 7,1 anos.

6. Sustentabilidade e governança (ESG)

6.1 Uso sustentável de energia

A modelagem evidencia que, sem intervenção, a demanda supera a geração no cenário moderado em ~7,1 anos. As ações prioritárias são:

- **Ampliação solar:** o módulo 4 tem consumo próprio de apenas 1 kW e capacidade máxima de 100 kW. Duplicar os painéis solares é a expansão de menor impacto estrutural.
- **Eficiência por módulo:** substituir o crescimento de consumo a 12 % por um perfil de 8 % (eficiência) adia o ponto crítico de 7,1 para mais de 20 anos.
- **Gestão de perda:** priorizar conexões de peso 1–2 para fluxos de alta potência reduz a perda de 13,9 % (peso 3) para menos de 5 % (peso 1).

6.2 Expansão organizada da colônia

A árvore de criticidade da Fase 3 continua como guia de expansão:

1. Novos módulos Vitais (habitação, suporte de vida) exigem redundância de conexão.
2. Módulos de Sustento podem ser adicionados via hubs existentes (Nuclear, Solar).
3. Módulos de Expansão (laboratório, oficina) devem ser acrescentados sem criar novos pontos de articulação — lição direta da análise de Tarjan.

6.3 Priorização de sistemas críticos

A análise combinada de centralidade e criticidade gera uma matriz de risco:

- **Alta centralidade + Alta criticidade:** Reator Nuclear (#5) — hub energético vital. Falha isolaria múltiplos módulos vitais. Candidato a redundância de conexão.
- **Alta centralidade + Baixa criticidade:** Armazenamento e Logística (#9) — único ponto de corte, mas tier Expansão. Paradoxo topológico: a criticidade operacional subestima a criticidade estrutural. Recomenda-se adicionar uma aresta alternativa conectando o Gerador Eólico diretamente à rede principal.
- **Baixa centralidade + Alta criticidade:** Suporte Médico (#7), Suporte de Vida (#2) — módulos vitais bem protegidos por múltiplos caminhos alternativos.

6.4 Governança tecnológica

O SIGIC implementa princípios de governança computacional responsável:

- **Determinismo e auditabilidade:** todas as análises são funções puras — dado o mesmo grafo, o mesmo resultado é produzido. Não há aleatoriedade oculta.
- **Separação de dados e I/O:** os algoritmos (BFS, DFS, Dijkstra, Tarjan, Brandes) nunca imprimem — devolvem dados. A apresentação fica exclusivamente no CLI. Isso facilita testes automatizados e auditoria independente.
- **Fonte única de verdade:** os módulos são definidos uma única vez (`aurora_siger.operations.MODULES`) e referenciados por todos os subsistemas. Não há tabelas paralelas que possam divergir.
- **Transparência de limitações:** o modelo exponencial $C(t) = C_0 e^{rt}$ assume taxa constante — uma simplificação. O SIGIC documenta esse pressuposto e oferece cenários alternativos (Seção 5.5) exatamente para mapear a sensibilidade a essa escolha.

6.5 Redução de desperdícios

A função `energy_loss_by_distance` e `optimize_energy_distribution` identificam os módulos com maior ineficiência de distribuição. Conexões de peso 3 (perda ~14 %) devem ser revistas ou complementadas por roteamento alternativo via nós intermediários mais próximos. A recomendação de curto prazo é reavaliar as arestas 1 – 12 (Controle – Laboratório) e 6 – 7 (Comunicações – Suporte Médico), que têm peso 3 e poderiam ser roteadas por caminhos de menor custo via Habitat (#3).

7. Conclusão

O SIGIC materializa a quarta fase do arco Aurora Siger — decolagem (Fase 1), pouso (Fase 2), operação (Fase 3) e agora **mapeamento topológico** (Fase 4). A contribuição conceitual desta fase é revelar que a mesma colônia que opera no tempo (Fase 3) também existe no espaço — como uma rede de dependências que a simulação horária não capturava.

A descoberta mais relevante é o paradoxo do ponto de articulação: o Armazenamento e Logística (#9), módulo de tier Expansão (prioridade 4), é o único ponto de corte da rede. Remover um módulo “de baixa prioridade”

isolaria o Gerador Eólico. A lição é que **criticidade operacional e criticidade topológica são dimensões ortogonais** — e um sistema de gestão completo precisa das duas.

A modelagem matemática adiciona uma terceira dimensão: o tempo. Com $C(t) = C_0 e^{rt}$ ancorado nos 80,5 kW iniciais e nos 210 kW instalados, o horizonte crítico de 7,1 anos não é uma abstração — é o prazo antes do qual a Aurora Siger precisa expandir sua geração ou contrair seu consumo. O ensaio complementar docs/fase-4/operacao-a-topologia.md desenvolve essa síntese narrativa.

Referências

- Repositório do portfólio (esta consolidação): <https://github.com/iurileo-hub/FIAP-Aurora-Siger>
- Especificação de design: docs/superpowers/specs/
- Ensaio reflexivo: docs/fase-4/operacao-a-topologia.md
- Diagrama da rede: fases/fase-4/figuras/rede_colonia.pdf
- Código-fonte do SIGIC: aurora_siger/colony/